

Rajjo Gujar 🍷 Discord Bot - Complete Documentation

Executive Summary

Rajjo Gujar 🍷 is a comprehensive, feature-rich Discord bot built with Python that combines AI-powered conversational capabilities with traditional Discord bot functionality. The bot leverages Large Language Model (LLM) integration via Ollama, provides music playback through Lavalink, includes moderation tools, fun games, and maintains comprehensive logging capabilities through SQLite database integration.

Table of Contents

1. [Project Overview](#)
2. [Architecture & Technology Stack](#)
3. [Core Features](#)
4. [File Structure Analysis](#)
5. [Installation & Setup Guide](#)
6. [Component Documentation](#)
7. [Database Schema](#)
8. [API Integration](#)
9. [Command Reference](#)
10. [Development Guidelines](#)
11. [Security Considerations](#)
12. [Troubleshooting](#)

Project Overview

Bot Identity and Purpose

Rajjo Gujar 🍷 is designed as a playful, confident, and witty female assistant character that provides both entertainment and utility functions for Discord servers. The bot maintains a consistent personality through carefully crafted system prompts while offering comprehensive server management capabilities.

Key Characteristics

- **Personality-Driven:** Maintains a consistent, engaging character with sass, humor, and charm
- **AI-Powered:** Integrates with Ollama for context-aware conversations
- **Modular Architecture:** Uses [Discord.py](#) cogs for organized, maintainable code
- **Comprehensive Logging:** All interactions logged to SQLite for analytics
- **Multi-Modal:** Supports text, voice, and image interactions

Architecture & Technology Stack

Core Technologies

Backend Framework

- **Python 3.10+**: Primary programming language
- **Discord.py 2.3.2**: Discord API wrapper for bot functionality
- **FastAPI**: LLM API backend server
- **Ollama**: Local LLM inference engine

Data Management

- **SQLite**: Local database for logging and persistence
- **aiosqlite 0.19.0**: Async SQLite operations
- **pandas 2.0.3**: Data manipulation and analysis

Media & Communication

- **Lavalink 4.0.4**: High-performance music playback
- **Google TTS (gTTS) 2.3.2**: Text-to-speech functionality
- **aiohttp 3.8.5**: Async HTTP client operations

Configuration & Utilities

- **python-dotenv 1.0.0**: Environment variable management
- **PyYAML**: System prompt configuration

Architecture Pattern

The bot follows a **modular cog-based architecture** where functionality is separated into distinct modules:

- **Main Controller** (`main.py`): Core bot initialization and cog management
- **Cogs**: Specialized modules for different functionality areas
- **Utilities**: Shared functions and database operations
- **LLM Server**: Separate FastAPI service for AI functionality

Core Features

1. AI Conversational System

Context-Aware Chat

- Maintains conversation history with context awareness
- Personality-driven responses via configurable system prompts
- Multi-turn conversation support with memory

Image Analysis

- Processes attached images using vision-enabled models
- Detailed image descriptions and analysis
- Base64 image encoding for API transmission

Text-to-Speech Integration

- Voice channel joining and speaking capabilities
- Google TTS integration for natural voice output
- Automatic audio streaming through Discord voice

2. Music Playback System

Lavalink Integration

- High-performance, low-latency music streaming
- Support for YouTube, SoundCloud, and direct URLs
- Advanced queue management with interactive controls

Interactive Controls

- Discord UI buttons for play/pause/skip/stop
- Real-time queue display and management
- Loop functionality and volume control

3. Moderation Tools

Message Management

- Bulk message purging with permission checks
- Message edit and delete logging
- Configurable log channels per server

Server Administration

- Giveaway system with interactive participation
- Channel-specific logging configuration
- Event-driven moderation actions

4. Entertainment & Games

Russian Roulette System

- Multiple game modes (normal, hard, hardcore)
- Interactive lobby system with join/leave mechanics
- Configurable chamber and bullet counts
- Turn-based gameplay with forfeit options

Meme Integration

- Reddit API integration for meme fetching
- Multiple sorting options (hot, top, new, etc.)
- Time-based filtering for content freshness

Roasting System

- Curated collection of clean, humorous roasts
- Multiple roast types including backhanded compliments
- User-friendly interface with embed formatting

5. Information & Utilities

Server Information

- Comprehensive server statistics and metadata
- User information with role and join date details
- Latency and performance monitoring

Utility Commands

- Markdown message posting to specific channels
- Interactive button demonstrations
- Help system with dynamic command listing

File Structure Analysis

```
Discord_Bot/
├── main.py           # Core bot initialization and startup
├── methods.py        # Conversation history and LLM utilities
├── requirements.txt   # Python dependencies
├── .env              # Environment variables (BOT_TOKEN, API_URL)
├── .gitignore        # Git ignore patterns
├── LICENSE           # MIT License
├── README.md         # Project documentation
├── data/             # Database and data storage
│   ├── chat_log.db   # SQLite database (auto-created)
├── LLM_server/       # FastAPI backend for AI
│   └── ollama_call.py # LLM inference endpoints
├── cogs/             # Modular functionality
│   ├── __init__.py   # Cogs initialization
│   ├── ai_chat.py    # AI conversation and TTS
│   ├── music.py      # Lavalink music playback
│   ├── moderation.py # Server moderation tools
│   ├── info.py       # Information and utility commands
│   ├── fun.py        # Games and entertainment
│   ├── events.py     # Event handlers
│   └── roast.py      # Roasting functionality
├── utils/            # Shared utilities
│   ├── utility.py    # Database logging decorators
│   └── sys_prompts.yaml # AI personality prompts
└── assets/           # Media and static files
```

```
|— lavalink/          # Lavalink configuration
|— token_counter/     # Token usage tracking
```

Installation & Setup Guide

Prerequisites

System Requirements

- Python 3.10 or higher
- pip package manager
- Git version control

External Services

- Discord Bot Token from Discord Developer Portal
- Ollama installation for LLM functionality
- Lavalink server for music playback

Step-by-Step Installation

1. Repository Setup

```
git clone https://github.com/AyushSharma297/Discord_Bot.git
cd Discord_Bot
git checkout features
```

2. Virtual Environment Creation

```
python -m venv discord_bot_env
source discord_bot_env/bin/activate # Linux/macOS
# or
discord_bot_env\Scripts\activate   # Windows
```

3. Dependency Installation

```
pip install -r requirements.txt
```

4. Environment Configuration

```
# Create .env file with the following variables:
BOT_TOKEN=your_discord_bot_token
API_URL=http://localhost:8000/ollama_query/
IMG_API_URL=http://localhost:8000/ollama_img/
```

5. Discord Bot Setup

- Create application at Discord Developer Portal

- Generate bot token and copy to .env file
- Enable Message Content Intent in bot settings
- Generate OAuth2 URL with appropriate permissions
- Add bot to target Discord server

6. Ollama Setup

```
# Install Ollama (varies by system)
# Pull required models
ollama pull llama3.1:latest
ollama pull gemma2:2b # For image analysis
```

7. Lavalink Configuration

- Download and configure Lavalink server
- Update connection details in [main.py](#)
- Ensure Java runtime environment is available

8. Launch Sequence

```
# Terminal 1: Start LLM Server
uvicorn LLM_server.ollama_call:app --host 0.0.0.0 --port 8000 --reload

# Terminal 2: Start Lavalink Server
java -jar Lavalink.jar

# Terminal 3: Start Discord Bot
python main.py
```

Component Documentation

Core Bot Controller ([main.py](#))

Primary Responsibilities

- Bot initialization with appropriate intents
- Database schema creation and management
- Cog loading and management system
- Lavalink connection establishment
- Global error handling

Key Components

Database Initialization

```
async def init_db():
    # Creates chat_log table with comprehensive logging fields
    # Handles database file creation and schema updates
```

Custom Voice Client

```
class LavalinkVoiceClient(discord.VoiceClient):  
    # Specialized voice client for Lavalink integration  
    # Handles voice state updates and server connections
```

Cog Management

```
async def load_cogs():  
    # Dynamic cog loading with error handling  
    # Supports hot-reloading for development
```

AI Chat System (cogs/ai_chat.py)

Core Functionality

Conversational AI

- Context-aware chat with conversation history
- Personality-driven responses via system prompts
- Multi-turn conversation support

Image Analysis

- Base64 image encoding and transmission
- Vision model integration for image description
- Comprehensive error handling for API failures

Text-to-Speech

- Voice channel connection management
- Google TTS integration with audio streaming
- Automatic voice state handling

System Prompt Management

The bot uses YAML configuration for personality prompts:

```
system_prompts:  
  - id: rp_v1  
    name: Rajjo Gujjar Personality Prompt  
    content: |  
      Act as Rajjo Gujjar 🇮🇳, a playful, confident, and witty female assistant...
```

Music System (cogs/music.py)

Architecture Overview

Lavalink Integration

- High-performance audio streaming

- Support for multiple audio sources
- Advanced queue management

Interactive Controls

```
class MusicControls(discord.ui.View):
    # Pause/resume functionality
    # Skip and stop controls
    # Queue display and management
    # Loop toggle functionality
```

Audio Source Support

- YouTube Music search integration
- SoundCloud compatibility
- Direct URL playback support
- Playlist loading capabilities

Moderation System (cogs/moderation.py)

Message Management

- Bulk message purging with permission verification
- Configurable purge limits and safety checks
- Administrator permission requirements

Logging System

- Message edit and delete event tracking
- Configurable log channels per server
- Comprehensive embed formatting for log entries

Giveaway System

```
class GiveawayView(discord.ui.View):
    # Interactive entry system
    # Automatic winner selection
    # Timeout handling and cleanup
```

Entertainment System (cogs/fun.py)

Russian Roulette Game

Game Modes

- Normal: 1 bullet, 6 chambers, spin each turn
- Hard: 2 bullets, 6 chambers, spin each turn
- Hardcore: 1 bullet, 6 chambers, no spinning (sequential)

Game Management


```
class RRGame:
    # Player management and turn tracking
    # Cylinder state management
    # Win condition evaluation
```

Interactive UI

```
class LobbyView(discord.ui.View):
    # Join/leave functionality
    # Host controls for game starting
    # Real-time player list updates
```

Reddit Meme Integration

- Multiple subreddit support
- Sorting options (hot, new, top, rising, controversial)
- Time filtering for top/controversial sorts
- Content filtering for appropriate material

Information System (cogs/info.py)

Server Information

- Comprehensive server statistics
- Member count and online status
- Boost information and verification levels
- Channel and role counts

User Information

- Profile details with avatar display
- Join dates and account creation
- Role listings with comprehensive formatting

Utility Commands

- Dynamic help generation from command metadata
- Latency monitoring with shard information
- Markdown message posting with embed options

Event System (cogs/events.py)

Member Events

- Welcome message system for new members
- Automatic DM delivery with error handling

Voice State Management

- Lavalink voice update handling
- Voice server update processing
- Seamless voice channel transitions

Utility System (utils/utility.py)

Database Logging

```
def log_command(func):  
    # Decorator for automatic command logging  
    # Captures user input and bot responses  
    # Comprehensive context preservation
```

Chat Logging

```
async def log_chat_to_db(ctx, user_message: str, bot_response: str):  
    # Direct chat interaction logging  
    # Supports AI conversation tracking  
    # Maintains conversation continuity
```

Database Schema

chat_log Table Structure

```
CREATE TABLE chat_log (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    time TEXT,                -- Timestamp of interaction  
    user TEXT,                -- Discord username#discriminator  
    user_message TEXT,        -- User input message  
    bot_response TEXT,        -- Bot response content  
    location TEXT,            -- Channel name with # prefix  
    channel_id INTEGER,       -- Discord channel ID  
    server TEXT               -- Server/guild name  
);
```

Data Flow

1. User interaction triggers command or chat
2. Context captured including user details, message content
3. Bot response generated and captured
4. Complete interaction logged to database
5. Historical context available for AI conversations

API Integration

LLM Server (LLM_server/ollama_call.py)

Text Query Endpoint

```
@app.post("/ollama_query/")
async def ollama_query(request: TextQueryRequest):
    # Processes text-based AI queries
    # Supports system and user prompts
    # Returns response with timing metrics
```

Image Query Endpoint

```
@app.post("/ollama_img/")
async def ollama_img(request: ImageQueryRequest):
    # Processes image analysis requests
    # Base64 image handling
    # Vision model integration
```

Health Check

```
@app.get("/health/")
async def health_check():
    # Service availability verification
    # System status monitoring
```

External API Integration

Reddit API

- Public JSON endpoints for meme fetching
- Rate limiting and error handling
- Content filtering for appropriate material

Discord API

- Full [Discord.py](#) integration
- Slash command support
- Interactive UI components (buttons, dropdowns)

Command Reference

AI Commands

Command	Description	Usage
<code>!chat <message>;</code>	Context-aware AI conversation	<code>!chat</code> How are you today?
<code>!speak [message]</code>	TTS in voice channel	<code>!speak</code> Hello everyone!
<code>!chat_img <prompt>;</code>	Image analysis with AI	Upload image + <code>!chat_img</code> What's in this picture?

Music Commands

Command	Description	Usage
<code>!play <query>;</code>	Play music from various sources	<code>!play</code> Never Gonna Give You Up
<code>!queue</code>	Show current music queue	<code>!queue</code>
<code>!skip</code>	Skip current track	<code>!skip</code>
<code>!pause</code>	Pause/resume playback	<code>!pause</code>
<code>!disconnect</code>	Disconnect from voice	<code>!disconnect</code>
<code>!nowplaying</code>	Show current track info	<code>!nowplaying</code>

Moderation Commands

Command	Description	Usage	Permissions
<code>!purge <count>;</code>	Delete messages	<code>!purge</code> 10	Manage Messages
<code>!setlog <channel>;</code>	Set log channel	<code>!setlog</code> mod-logs	Administrator
<code>!unsetlog</code>	Remove log channel	<code>!unsetlog</code>	Administrator
<code>!giveaway <time>; <prize>;</code>	Start giveaway	<code>!giveaway</code> 60 Discord Nitro	Manage Guild

Fun Commands

Command	Description	Usage
<code>!roulette [mode] [ante]</code>	Start Russian Roulette game	<code>!roulette</code> hard 100
<code>!meme [sort] [time]</code>	Get Reddit meme	<code>!meme</code> hot day
<code>!roast [@user] [type]</code>	Roast user playfully	<code>!roast</code> @user compliment

Information Commands

Command	Description	Usage
<code>!help</code>	Show all commands	<code>!help</code>
<code>!ping</code>	Check bot latency	<code>!ping</code>
<code>!userinfo [@user]</code>	Show user information	<code>!userinfo @user</code>
<code>!serverinfo</code>	Show server details	<code>!serverinfo</code>
<code>!markdown <channel> [embed] <text></code>	Post markdown message	<code>!markdown general true</code> Bold text

Development Guidelines

Code Organization

Cog Structure

- Each cog represents a functional domain
- Self-contained with minimal external dependencies
- Consistent error handling and logging
- Documentation for all public methods

Database Operations

- Async operations throughout
- Proper connection handling with context managers
- Error handling and transaction management
- Data validation and sanitization

Error Handling

- Graceful degradation for API failures
- User-friendly error messages
- Comprehensive logging for debugging
- Recovery mechanisms where appropriate

Best Practices

Security

- Environment variable usage for sensitive data
- Permission checking for administrative commands
- Input validation and sanitization
- Rate limiting considerations

Performance

- Async/await patterns throughout
- Connection pooling for database operations
- Efficient caching strategies
- Resource cleanup and proper disposal

Maintainability

- Modular architecture with clear separation of concerns
- Comprehensive documentation and comments
- Consistent coding style and conventions
- Version control with meaningful commit messages

Security Considerations

Token Protection

- Discord bot token stored in environment variables
- .env file excluded from version control
- No hardcoded secrets in source code

Permission Management

- Role-based access control for commands
- Permission verification before sensitive operations
- Administrator-only access for server configuration

Input Validation

- Message content sanitization
- File type verification for image uploads
- SQL injection prevention through parameterized queries

API Security

- Rate limiting on external API calls
- Error handling to prevent information disclosure
- Timeout management for external requests

Troubleshooting

Common Issues

Bot Not Responding

- Verify bot token is correct and valid
- Check if Message Content Intent is enabled
- Ensure bot has necessary permissions in target channels

Music Not Playing

- Confirm Lavalink server is running and accessible
- Verify bot has Connect and Speak permissions
- Check if user is in a voice channel

AI Chat Not Working

- Ensure Ollama server is running on specified port
- Verify required models are pulled and available
- Check API_URL environment variable

Database Errors

- Verify data directory exists and is writable
- Check SQLite file permissions
- Ensureaiosqlite dependency is installed

Performance Optimization

Memory Management

- Regular cleanup of conversation history
- Efficient caching of frequently accessed data
- Proper disposal of resources after use

Network Optimization

- Connection pooling for API requests
- Efficient data serialization and transmission
- Proper timeout handling for external services

Monitoring and Logging

Log Analysis

- Comprehensive logging throughout application
- Structured logging for easier analysis
- Performance metrics collection

Health Monitoring

- Regular health checks for external dependencies
- Automated alerts for service failures
- Performance monitoring and optimization

This documentation provides a comprehensive overview of the Rajjo Gujar – Discord Bot, covering all aspects from installation to advanced usage and maintenance. The bot represents a sophisticated integration of modern AI capabilities with traditional Discord bot functionality, providing both entertainment and utility value for Discord communities.